Project 12 — Novelty & Implementation (Simple English)

Based on PPT.pdf

🌟 What Makes This Project *Novel*? (The "So What?" Answer)

Novelty = what your project does that NO ONE has done before (or done completely).

Here's the honest breakdown. Every paper before yours did *one thing well* but missed the rest:

SHAP/LIME	✅ Explains predictions	❌ Only correlation	❌ No recourse
Wachter (2017)	✅ Gives "what if"	❌ Impossible changes	❌ No causal logic
Ustun (2019)	✅ Actionable recourse	❌ Linear models only	❌ No causation
Karimi (2021)	✅ Causal recourse theory	❌ No real UI	❌ Too slow
DiCE (2020)	✅ Multiple options	❌ Causally wrong	❌ Ignores dependencies

YOUR project does ALL of this together:

> ✅ **Causal explanations** (not just correlational like SHAP) ✅ **Actionable recourse** that is **causally feasible** (not impossible like Wachter) ✅ **Works on any ML model** (not just linear like Ustun) ✅ **Filters out features you can't change** (age, gender — immutable) ✅ **Deployed as a real, usable dashboard** (not just a research paper like Karimi)

🗣️ How to Say "Novelty" in Your Viva (Memorize This)

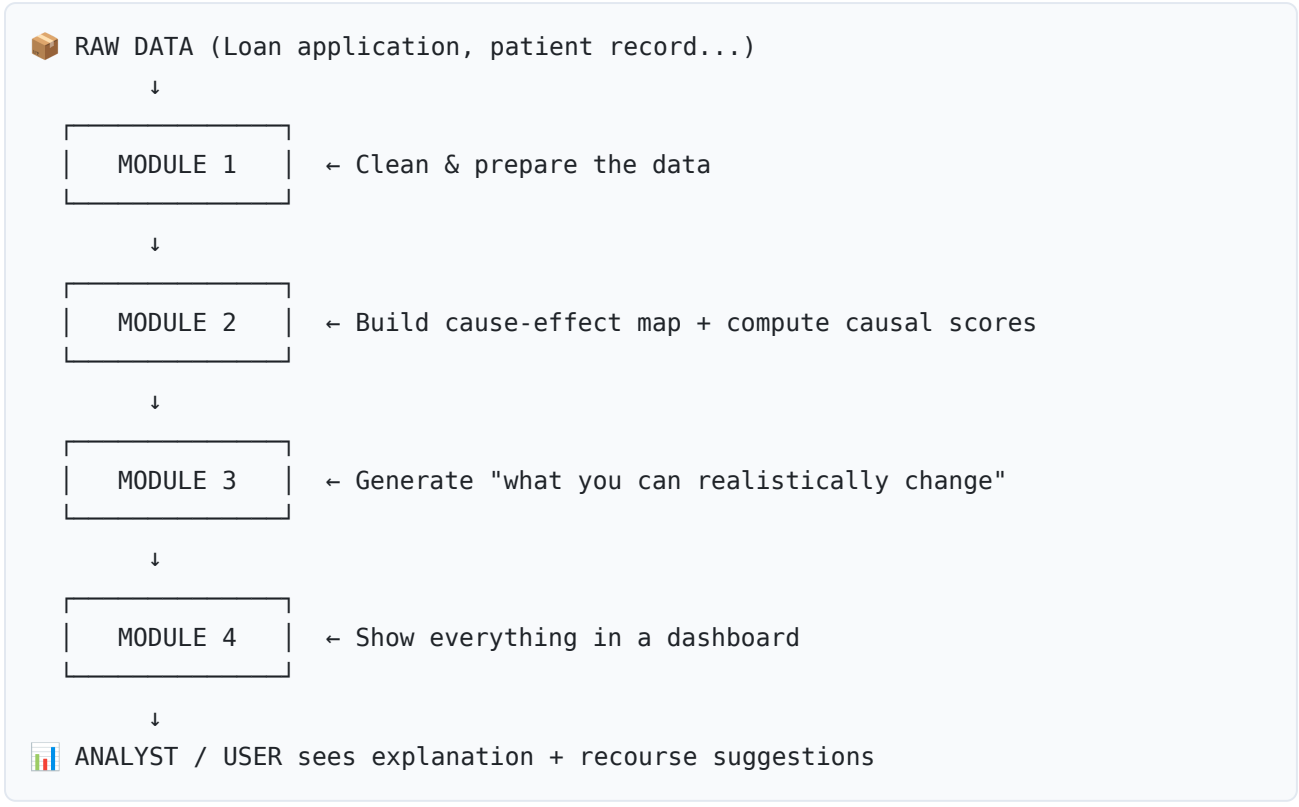
> *"Sir, the novelty of our project is three-fold. First, we use Structural Causal Models instead of correlational methods like SHAP and LIME — so our explanations reflect true cause-and-effect, not just patterns. Second, we generate actionable recourse that is causally consistent — meaning if you change one feature, the downstream effects on related features are automatically accounted for. Third, unlike prior research which was purely theoretical, we deploy this as a working decision-support dashboard with regulatory audit logging — making it production-ready."*

🔍 The 4 Problems We're Solving (From the PPT)

Problem	What Exists Today	Why It's Bad
Decision Transparency	Complex ML models give no explanation	Users don't trust the AI; no accountability
Causal Validity	SHAP & LIME show correlations only	Explanations can point to wrong features
Actionability	Counterfactual methods suggest unrealistic changes	"Reduce your age by 3 years" — useless!
Feature Constraints	Mutable vs. immutable features not separated	AI suggests changing things you literally cannot change

🏗️ How We're Building It — The 4 Modules Explained Simply

Think of the system like a **factory assembly line**. Raw data goes in one end, and a useful, trustworthy decision explanation comes out the other end.



MODULE 1 — Data Ingestion & Preprocessing

What it does in simple words:

> *Clean up the raw data so the AI can work with it.*

Steps:

- Take in tabular data — e.g., a bank loan application form (name, income, age, credit score, savings, etc.)
- Check for errors, missing values, wrong data types → fix them
- Normalize numbers (so ₹50,000 income and 700 credit score are on the same scale)
- **Label each feature:** Is this something the person CAN change (mutable) or CANNOT change (immutable)?

Mutable ✅ (Can Change)	Immutable ❌ (Cannot Change)
Income	Age
Savings	Gender
Credit Score	Date of Birth
Loan Amount Requested	Nationality

Why this matters: If we don't separate these upfront, the AI might later suggest "increase your age" — which is nonsense.

MODULE 2 — Causal Discovery & Attribution Engine (The Brain)

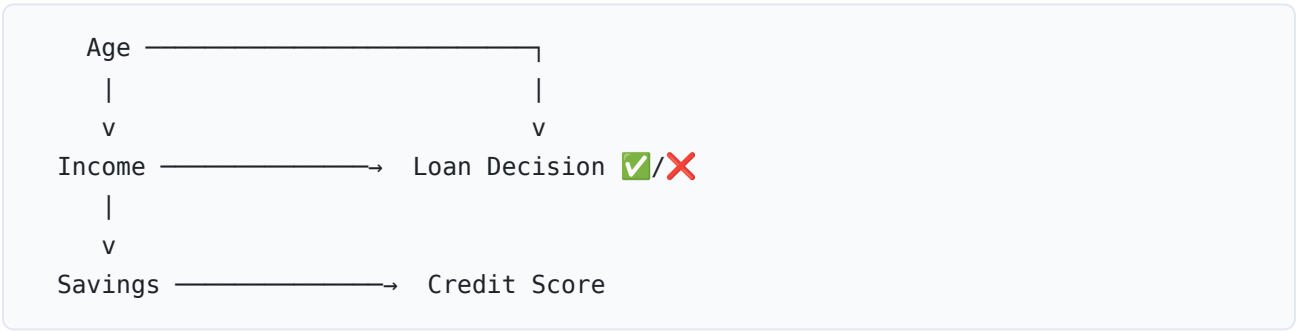
What it does in simple words:

> *Draw a map of what causes what — then figure out which causes actually drove the AI's decision.*

Step 2a — Build the Causal Map (DAG)

We use algorithms to learn a **Directed Acyclic Graph (DAG)** from the data.

Simple example DAG:



This tells us: Age affects Income, Income affects both Savings and the Loan Decision, Savings affects Credit Score.

Two algorithms we use:

- **PC Algorithm** — works by checking which variables are statistically independent of each other and building the graph from those checks (like a detective eliminating suspects)
- **NOTEARS** — a newer, faster method that uses calculus (gradient descent) to find the best causal graph automatically

Step 2b — Compute Causal Attribution (do-calculus)

Now we use **Pearl's do-calculus** — specifically the `do()` operator — to ask:

> *"If I FORCE this feature to a new value (intervene), what happens to the final decision?"*

This is different from SHAP which asks "what value did this feature have?" Our method asks "what if we actually **changed** it?"

Output of Module 2:

- A bar chart showing each feature's TRUE causal impact on the decision
- A visual causal graph highlighting which path drove the outcome

MODULE 3 — Recourse Engine & Ranking

What it does in simple words:

> *"You were rejected. Here's exactly what you can realistically change to get approved."*

How it works:

It runs a **constrained optimization problem** — fancy math that finds the minimum-effort combination of feature changes that flips the decision from ❌ to ✅.

Rules it follows:

1. ❌ Don't touch immutable features (age, gender)
2. ✅ Only change mutable features
3. 🔗 Respect the causal chain — if income increases, savings estimate also updates accordingly
4. 📊 Check if the suggestion is realistic (within the range of real data)
5. 💰 Rank suggestions by "cost" — easiest/cheapest first

Example output:

```
🔴 Current Status: LOAN REJECTED

Recourse Options (ranked by effort):
──────────────────────────────────────
Option 1 (Easiest):
  → Increase monthly income by ₹6,000
  → Estimated time: 3–6 months
  → Predicted outcome: ✅ APPROVED

Option 2 (Alternative):
  → Pay off ₹25,000 in existing debt
  → Predicted outcome: ✅ APPROVED
──────────────────────────────────────
```

The **library used for this optimization** is **SciPy** — a Python math library that solves these kinds of constrained problems.

MODULE 4 — Decision-Support Dashboard (What Users See)

What it does in simple words:

> *Show everything — prediction, explanation, recourse — in a simple, visual, clickable interface.*

Built with Streamlit (for demo) / React.js (for production)

What the dashboard shows:

- 🎯 **Prediction result** — Approved / Rejected with confidence %
- 📊 **Causal attribution chart** — bar chart showing which features causally drove the decision
- 🗺️ **Interactive DAG** — a visual map of the cause-effect relationships, with the influential paths highlighted
- 💡 **Recourse panel** — ranked list of "what you can change"
- 📋 **Audit log** — every decision recorded in PostgreSQL for regulatory compliance (e.g., GDPR, RBI guidelines)

🛠️ Full Tech Stack (From the PPT) — Explained Simply

What It Is	Tool Used	Why We Use It (Simple)
Programming Language	Python 3.10+	Everything is built in Python — easy, powerful, huge ML ecosystem
Data Handling	NumPy, Pandas	NumPy for math, Pandas for tables — like Excel but in code
ML Model	Scikit-Learn (Random Forest, Logistic Regression)	Trains the base AI that makes predictions (approved/rejected)
Causal Inference	**DoWhy, causal-learn**	DoWhy = Microsoft's library to build SCMs and compute causal effects; causal-learn = learns DAGs from data
Optimization (Recourse)	**SciPy**	Solves the math problem of "minimum cost changes" — finds the cheapest recourse
Explainability (Comparison)	SHAP, Matplotlib	We also compute SHAP scores to *compare* — to show that causal > correlational
Backend API	**FastAPI**	The "waiter" between the AI and the dashboard — exposes prediction/recourse as web services
Frontend Dashboard	**Streamlit**	Builds the visual interface where analysts see everything
Database	**PostgreSQL**	Stores all decisions, predictions, and recourse suggestions — for audit/compliance

🔑 The Two Libraries That Make This Project Unique

1. DoWhy (by Microsoft)

This is the **heart** of the causal engine.

- Lets you define a causal graph (DAG)
- Computes causal effects using the `do()` operator
- Tests whether the causal effect is robust (refutation tests)
- Industry-grade — actually used at Microsoft for real decisions

> **Simple analogy:** DoWhy is to causality what Scikit-Learn is to machine learning — it's the standard library.

2. causal-learn

- Automatically **learns the DAG from your data** using the PC Algorithm
- You don't have to manually draw the cause-effect map — it discovers it

> **Simple analogy:** If DoWhy is the GPS navigator, causal-learn is the map-making tool that creates the map from scratch by observing traffic patterns.

🗣️ How to Explain Implementation in Viva

If asked "How does your system work?"

> *"Sir, the system has four modules. First, we preprocess the tabular data and tag features as mutable or immutable. Second, we learn a causal graph using causal-learn's PC Algorithm, and compute true causal attributions using DoWhy and do-calculus. Third, SciPy solves a constrained optimization to find the minimum-cost feature changes that flip the decision — filtered to only mutable features. Finally, FastAPI exposes these as REST endpoints, and Streamlit renders everything in an interactive dashboard with full audit logging in PostgreSQL."*

If asked "Why Python?"

> *"Python has the richest ecosystem for ML and causal inference. DoWhy, causal-learn, XGBoost, SHAP — all are Python-first libraries. It allows us to prototype fast and deploy with minimal overhead."*

If asked "Why FastAPI over Flask?"

> *"FastAPI is asynchronous, auto-generates API documentation, and is significantly faster for handling concurrent requests — which matters when multiple analysts are using the dashboard simultaneously."*

🧠 One-Page Mental Model (Stick This in Your Head)

```
The Problem:
  AI rejects your loan → you don't know why → you don't know what to do

What SHAP does:
  "These features were important" → still just correlations → useless for action

What Counterfactuals do:
  "Change your age by -3 years" → impossible → still useless

What YOUR project does:
  Step 1: Learn HOW features cause each other (DAG via causal-learn)
  Step 2: Compute WHAT caused the rejection (do-calculus via DoWhy)
  Step 3: Find the CHEAPEST REALISTIC CHANGE to get approved (SciPy optimization)
  Step 4: Show it all in a DASHBOARD (Streamlit + FastAPI + PostgreSQL)

Result: "Increase your income by ₹6,000/month OR pay off ₹25,000 debt → APPROVED"
That's it. That's the project.
```

> IMPORTANT
>
> The **three words** that define this project's novelty: **Causal. Feasible. Actionable.** — Causal (not correlational), Feasible (respects reality), Actionable (tells you exactly what to do). Use these three words in your viva.